Nonlinear Dyn
DOI 10.1007/s11071-014-1492-0

ORIGINAL PAPER

A fast color image encryption algorithm based on hyper-chaotic systems

Benyamin Norouzi · Sattar Mirzakuchaki

Received: 22 April 2013 / Accepted: 26 May 2014

Abstract This paper presents a new way of image encryption scheme, which consists of two processes; key stream generation process and one-round diffusion process. The first part is a pseudo-random key stream generator based on hyper-chaotic systems. The initial conditions for both hyper-chaotic systems are derived using a 256-bit-long external secret key by applying some algebraic transformations to the key. The original key stream is related to the plain-image which increases the level of security and key sensitivity of the proposed algorithm. The second process employs the image data in order to modify the pixel gray-level values and crack the strong correlations between adjacent pixels of an image simultaneously. In this process, the states which are combinations of two hyper-chaotic systems are selected according to image data itself and are used to encrypt the image. This feature will significantly increase plaintext sensitivity. Moreover, in order to reach higher security and higher complexity, the proposed method employs the image size in key stream generation process. It is demonstrated that the number of pixel change rate (NPCR) and the unified average changing intensity (UACI) can satisfy security and performance requirements (NPCR >99.80 %, UACI >33.56 %) in one round of diffusion. The experimental results reveal that the new image encryption algorithm has the advantages of large key space, high security, high sensitivity, and high speed. Also, the distribution of gray-level values of the encrypted image has a semi-random behavior.

Keywords Color image encryption · Diffusion · Hyper-chaotic systems

B. Norouzi (✉) · S. Mirzakuchaki
Electronic Research Center, School of Electrical Engineering, Iran University of Science and Technology, P.O. Box 16846-13114, Tehran, Iran
e-mail: Benyamin_Norouzi@elec.iust.ac.ir

S. Mirzakuchaki
e-mail: M_Kuchaki@iust.ac.ir

1 Introduction

Along with the fast development of electronic data exchange, it is essential to protect the confidentiality of data from illegal access. Security breaches may affect user's privacy and reputation. So, data encryption is a useful means for ensuring reliable security in open networks such as the internet. The data include text, audio, image, and other multimedia data. Each type of data has its own characteristics; as a result, different methods should be employed to protect confidential image data from unauthorized access. Security of image and video data has become more and more important for many applications such as internet-based communications, video conferencing, secure facsimile, medical, and military applications.

Most of the available encryption schemes are used for text data. However, due to some inherent features of images such as bulk data capacity, strong correlation among pixels, and high computation complexity,

Published online: 24 June 2014

the algorithms that are appropriate for textual data may not be proper for multimedia data, especially under the scenario of online communications. Consequently, during the last few years, several image encryption algorithms have been proposed. Such encryption schemes are based on iterative fractional Fourier transform [1], Phase Encoding Algorithm in the Fresnel Transform Domain [2], the hyper-chaotic systems [3], and chaos maps with total shuffling [4]. The cryptosystems based on chaos are suitable for the secure transmission of images due to the intrinsic characteristics of chaotic systems such as ergodicity, randomness, simplicity, sensitivity to initial conditions, and system parameters. There exist three main types for image encryption based on chaos, namely position permutation (confusion), the value transformation (diffusion), and their compounding form [4–6]. The aim of permutation (confusion) phase was to disturb the strong correlation among pixels but due to the fact that the pixel value remains constant, the histogram of the encrypted image and the original image is identical, so its security is threatened by the statistical attacks. Arnold cat map is often used to shuffle the positions of the pixels in many cryptosystems [6–9], but the map has two weaknesses. One is that the iteration times are very limited, usually less than 1,000 times; the other is that the width and height of the plain image must be equal, or the image cannot be permuted directly. In the diffusion phase, the pixel values of the original image are modified sequentially by chaotic sequences such that a tiny change in the gray-level value of only one pixel is spread out to all the pixels (avalanche effect) [10,11]. Most of these diffusion schemes directly implement encryption by overlaying a chaotic sequence generated by a single chaotic map and the pixel value from the image. Compared to the confusion, diffusion may lead to higher security. In practice, confusion and diffusion are often combined in order to get high computational security [4,10]. The whole confusion-diffusion round repeats for a number of times to achieve a satisfactory level of security.

Applying more than one chaotic map can achieve better key space. For grayscale image, most algorithms use the one-dimension chaotic maps [5], but their limitations, such as small key space and weak security, are also disturbing. The purpose of using multiple chaotic maps and hyper-chaotic systems is to enlarge the key space to resist various attacks. Liu et al. [12] have used multiple chaotic maps to encrypt images. Although these algorithms have large key space and high sensitivity, their security is not high enough. Some researchers have investigated image encryption based on hyper-chaos [11,13,14], but in Ref. [15,16], some attacks were suggested to break a chaotic cryptosystem presented in Ref. [14]. It has been shown that this algorithm is weak against chosen-ciphertext and chosen-plaintext attacks.

This paper aims to overcome these weaknesses by designing a novel image encryption method with high security and high sensitivity. In order to fast decorrelate relations among pixels, two hyper-chaotic systems are employed to modify the pixel values and break the correlations between adjacent pixels of an image simultaneously in only one round of diffusion process. Based on the sequence which is generated by hyper-chaotic system and correlating keys with plaintext of an image to be encrypted, the algorithm uses different key streams when encrypting different input images (even with the same sequence based on hyper-chaotic system). These features strengthen the cryptosystem security. Experimental results and performance analysis prove the viability of this cryptography based on privacy, integrity, and authenticity.

The rest of this paper is organized as follows. Section 2 gives a brief introduction of pseudo-random number generators and hyper-chaotic systems. Section 3 introduces the proposed encryption scheme. Section 4 evaluates the security of the proposed algorithm. Some conclusions are drawn in Sect. 5.

2 Pseudo-random number generators

Random number generators are commonly used in encoding algorithms which can be classified into three classes: true random number generators, pseudo-random number generators (PRNG), and hybrid random number generators. Pseudo-Random Number Generators are cryptographic and deterministic algorithms used to produce numbers from an initial seed that must appear random but are necessarily predetermined. As we know, each dynamical system has specific characteristics. A good PRNG must have some properties such as high value of entropy, high dimensionality of the parent dynamical system, very large period of the generated sequence, good distribution, long period, portability, and so on [17,18]. In fact, the differences between discrete dynamical systems arise from these properties.

A probabilistic dynamical system is characterized as ergodic or non-ergodic by its marginal probability distributions. If the distributions have finite variances, so that a time-independent process-mean can be clearly defined, the system is called ergodic [19]. Moreover, the signs of Lyapunov exponents provide a good classification of the dynamics of PRNG systems that explain the separation rate of systems whose initial conditions differ by a small perturbation. Increasing the values and the number of positive Lyapunov exponents makes the probability distributions of the output chaotic sequences more homogeneous and reduces the correlations of chaotic outputs for different times and different space units [20–22].

One-dimensional chaotic systems such as Logistic map can be used as a pseudo-random number generator but their weaknesses, such as small key space and weak security (it generates sequences of extremely short period), are also disturbing [11,19].

Chen [23], Qi [24], and Lu [25] introduced three-dimensional autonomous chaotic systems that exhibit a chaotic behavior with a single positive Lyapunov exponent.

In order to increase the sensitivity, we seek PRNG systems with an improved random nature, or in other words, with more and higher Lyapunov exponents. Therefore, in this paper, we have employed hyper-chaotic systems. It is worth mentioning that according to Ref. [25–27], 3D systems proposed by chen, Qi and Lu are not hyper-chaotic. Because in order to obtain hyper-chaos, the following two basic conditions must be met:

(1) The minimal dimension of the phase space of an autonomous system is at least four which requires the minimum number of coupled first-order autonomous ordinary differential equations to be four.
(2) The number of terms in the coupled equations giving rise to instability is at least two, of which at least one has a nonlinear function. In other words, the system has at least two positive Lyapunov exponents satisfying the condition that the sum of all Lyapunov exponents is less than zero.

In recent years, hyper-chaotic systems have been extensively studied because of their ability to exhibit at least two positive Lyapunov exponents. In other words, the dynamics of the system expand in more than one direction and produce a much more complex attractor compared with the chaotic system with only one positive Lyapunov exponent. It means that hyper-chaotic systems generate better dynamical behaviors compared to chaotic systems. There are several 4D hyper-chaotic systems discovered in the high-dimensional social and economical systems such as Rossler system, Lorenz-Haken system, Chua's circuit, and Chen system [25].

In the suggested encryption scheme, the Chen's chaotic system is one that is employed in key scheming which is described as follows [27]:

$$\begin{cases} \dot{x}_1 = a\,(x_2 - x_1)\,, \\ \dot{x}_2 = -x_1 x_3 + d x_1 + c x_2 - x_4, \\ \dot{x}_3 = x_1 x_2 - b x_3, \\ \dot{x}_4 = x_1 + k, \end{cases} \tag{1}$$

where a, b, c, d, and k are system parameters. When $a = 36$, $b = 3$, $c = 28$, $d = -16$, and $-0.7 \leq k \leq 0.7$, the system is hyper-chaotic and can generate four hyper-chaotic sequences. We have used 0.2 for k in our calculations. The chaotic behaviors of this system are shown in Fig. 1.

Another chaotic system in our scheme is described as follows [26]:

$$\begin{cases} \dot{x}_5 = p\,(x_6 - x_5) + x_6 x_7, \\ \dot{x}_6 = q x_5 - x_6 - x_5 x_7 + x_8, \\ \dot{x}_7 = x_5 x_6 - r x_7, \\ \dot{x}_8 = s x_8 - x_5 x_7, \end{cases} \tag{2}$$

where p, q, r, and s are system parameters. When $p = 35$, $q = 8/3$, $r = 55$, and $s = 1.3$, the hyper-chaotic system is in the chaotic state and can produce four hyper-chaotic sequences. Here, we take the fourth-order Runge-Kutta (we have used the algorithm given in Matlab as ODE45 function.) method to solve the Eqs. (1) and (2) and obtain the sequences $x_1, x_2, \ldots, x_8$. These hyper-chaotic sequences are non-periodical and very sensitively dependent on initial conditions. The projections of the attractors are shown in Fig. 2.

3 Color image encryption algorithm

Without loss of generality, we suppose that the size of the color plain-image RGB_Image is $3 \times M \times N$. We transform RGB_Image into its R, G, and B components as shown in Fig. 3; the size of each color's (R, G, or B) matrix is $M \times N$, and the pixels' values range from 0 to

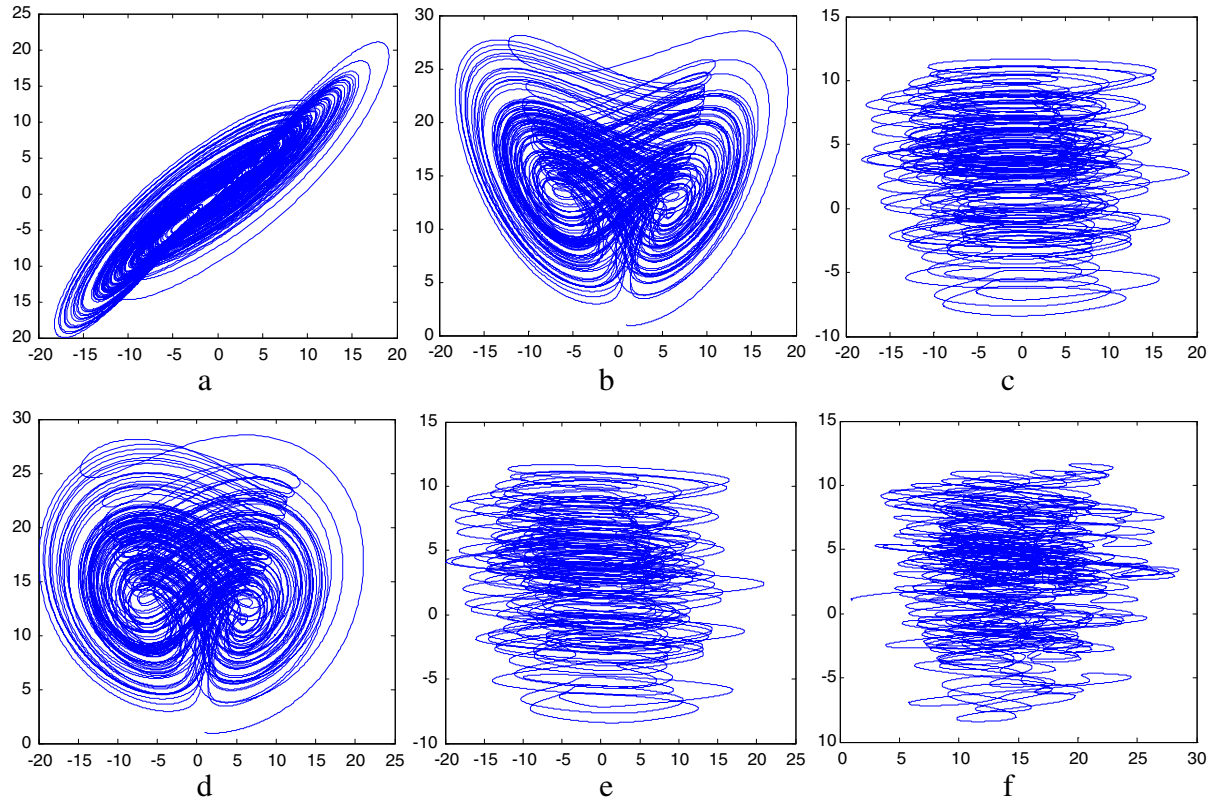

Fig. 1 The attractors of Chen's hyper-chaos system. **a** $x_1 - x_2$ plane; **b** $x_1 - x_3$ plane; **c** $x_1 - x_4$ plane; **d** $x_2 - x_3$ plane; **e** $x_2 - x_4$ plane; **f** $x_3 - x_4$ plane

255. Afterwards, each color's matrix (R, G or B) is converted into a one-dimensional vector of integers within $\{0, 1, \ldots, 255\}$. Each vector has a length of $L = N \times M$. The image encryption algorithm includes generating original chaotic key stream and one round of diffusion operation. Two hyper-chaotic systems which are described in Eqs. (1) and (2) are employed to produce the original chaotic key stream. The image encryption algorithm is based on the final secret key stream which is not only related to the original chaotic key stream but also to the original image. The image encryption algorithm includes generating original chaotic key stream and one round of diffusion process.

3.1 Key stream generation process

This process generates pseudo-random key streams for encryption of the color image based on the two hyper-chaotic systems. To this end, $8 \times L$ pseudo-random numbers are produced for each color image of size $M \times N$. Only $2 \times L$ pseudo-random numbers are used for the encryption of each component (R, G, or B).

3.1.1 Generation the initial conditions

Our image encryption algorithm employs a 256-bit-long external secret key (K). This key is segmented into 32-bit -long blocks (K_i) referred to as session keys. Such session keys are then utilized for the generation of the initial conditions of the hyper-chaotic systems described in Eqs. (1) and (2). The 256-bit-long external secret key (K) is given by $K = K_1, K_2, \ldots, K_8$.

To compute the initial conditions $\{x1_0, x2_0, x3_0, x4_0, x5_0, x6_0, x7_0, x8_0\}$ of the hyper-chaotic systems, four bytes K_{ij} are produced from session keys as shown in Fig. 4 ($i = 1, \ldots, 8$ and $j = 1, \ldots, 4$). From these K_{ij}, this method generates eight numbers (of 32 bits) Q_i as shown in Fig. 5. We have $k_{\text{sum}} = \text{mod}(\sum k_{ij}, 256)$.

The initial conditions are then obtained as follows:

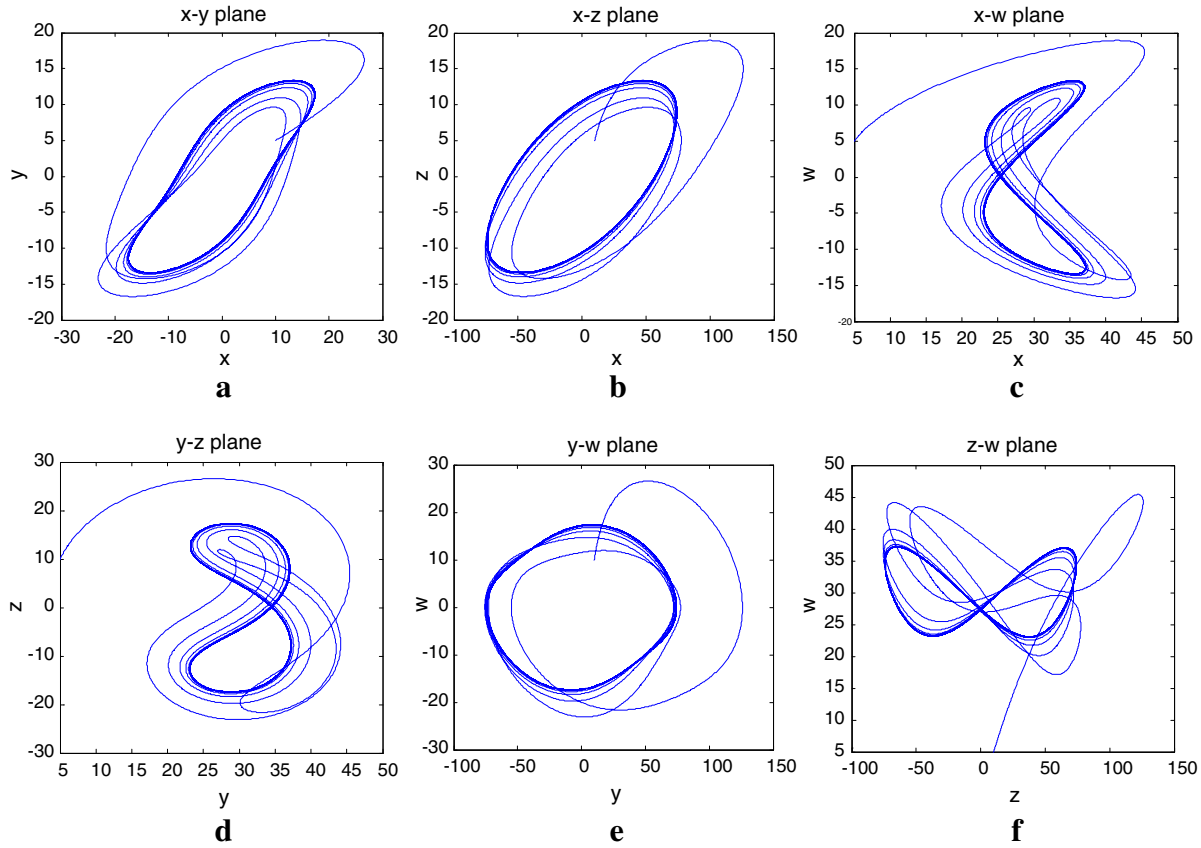

Fig. 2 The attractors of hyper-chaos system described in Eq. (2). **a** $x_5 - x_6$ plane; **b** $x_5 - x_7$ plane; **c** $x_5 - x_8$ plane; **d** $x_6 - x_7$ plane; **e** $x_6 - x_8$ plane; **f** $x_7 - x_8$ plane

$$Q_i = \text{bi2de}\,(Q_{i1}) \times 10^9 + \text{bi2de}\,(Q_{i2}) \times 10^6 + \text{bi2de}\,(Q_{i3}) \times 10^3 + \text{bi2de}\,(Q_{i4})\,. \quad (3)$$

$$xi_0 = \frac{Q_i}{L^2} \quad \text{for} \quad i = 1, \ldots, 8. \quad (4)$$

The function bi2de(x) converts binary value x to decimal number.

Obviously, this section shows that the initial conditions of the hyper-chaotic systems are greatly sensitive to the change in even a single bit of the 256-bit external secret key and the length of the plaintext (L); hence, the proposed algorithm with total complexity of 2^{256} can resist any brute-force attack.

The hyper-chaos systems which are shown in Eqs. (1) and (2) are employed to produce the original chaotic key stream as follows:

Step 1 using Runge-Kutta step size 0.001 and initial conditions $\{x1_0, x2_0, x3_0, x4_0, x5_0, x6_0, x7_0, x8_0\}$, iterate the hyper-chaotic systems for N_0 times to avoid the harmful effect of transitional procedure.

Step 2 two hyper-chaotic systems are iterated for L times continuously. For each iteration, we can get eight values $x_1, x_2, \ldots, x_8$. In other words, eight sequences $\{x_i\}$, $i = 1, 2, \ldots, 8$ will be produced. The size of each sequence is L.

Step 3 these decimal values are determined first as follows:

$$x_i = \text{floor}\left(\text{mod}\left(x_i \times 10^{14}, 256\right)\right), \quad (5)$$

where $i = 1, 2, \ldots, 8$; floor(x) returns the value of x to the nearest integer less than or equal to x. Mod (x, y) returns the remainder after division. The size of each sequence (x_i) is L. To make the states of the two chaoticsystems

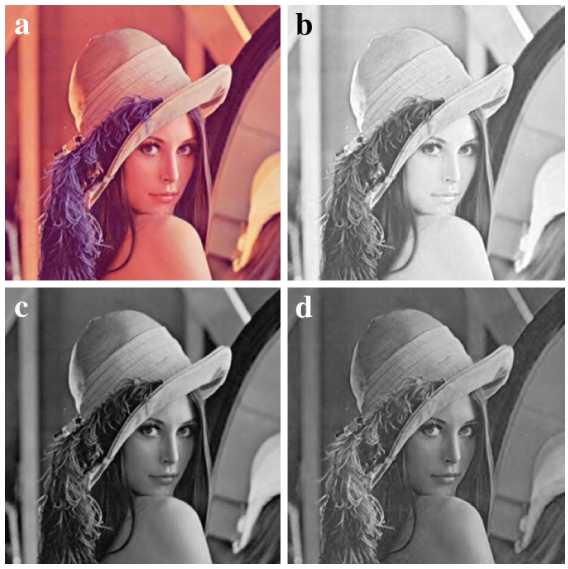

Fig. 3 Color plain-image and its R, G, and B components; **a** Lena color plain-image; **b** R component of plain-image; **c** G component of plain-image; and **d** B component ofplain-image

K_i (32 bit)			
K_{i1}	K_{i2}	K_{i3}	K_{i4}
8 bit	8 bit	8 bit	8 bit

Fig. 4 Generation of K_{ij} from the session keys

correlative, let

$$\begin{aligned} x_8 &= x_8 \oplus x_1, \\ x_7 &= x_7 \oplus x_2, \\ x_6 &= x_6 \oplus x_3, \\ x_5 &= x_5 \oplus x_4, \end{aligned} \tag{6}$$

The image encryption algorithm is based on the final secret key stream which is not only related to the original chaotic key stream but also to the plain-image.

3.2 Encryption algorithm

Our image encryption algorithm is based on the combination of state variables of two hyper-chaotic systems. Eight of the variables are combined differently, which may produce eight different combinations table, which is given in Table 1. Then, the encryption algorithm is given as follows:

Step 1 transform the color image RGB_image into three matrices R, G, and B, where the size of each matrix is $M \times N$ and the pixels' values range from 0 to 255.

Step 2 convert each color's matrix (R, G, or B) into a one-dimensional vector denoted by $R = \{r_1, r_2, \ldots, r_L\}$, $G = \{g_1, g_2, \ldots, g_L\}$, and $B = \{b_1, b_2, \ldots, b_L\}$.

Step 3 compute the sum of color's matrix pixels according to the following:

$$\text{sum.image} = \sum_{i=1}^{L} r_i + \sum_{i=1}^{L} g_i + \sum_{i=1}^{L} b_i. \tag{7}$$

Let $i \leftarrow 1$. The initial cr_{-1}, cg_{-1}, and cb_{-1} are set as follows:

$$a = \text{sum.image} - (r\,(1) + g\,(1) + b\,(1)), \tag{8}$$

$$\begin{aligned} cr_{-1} &= cg_{-1} = cb_{-1} \\ &= \text{floor}\left(\text{mod}\left(\frac{a}{256^5} \times 10^{10}, 256\right)\right). \end{aligned} \tag{9}$$

Remark 1 Three initial values for the diffusion function are deduced from Eq. (9) for encrypting the R, G, and B components of an image. These initial values are the summation of all pixels except for the first pixel components as given by Eq. (8), which is then used for the encryption of the first pixels. If the first pixel values are used in Eq. (9), however, due to the change in

Q_1	$\rightarrow$	$Q_{11} = k_{11} \oplus k_{84} \oplus k_{sum}$	$Q_{12} = k_{12} \oplus k_{83} \oplus k_{sum}$	$Q_{13} = k_{42} \oplus k_{52} \oplus k_{sum}$	$Q_{14} = k_{44} \oplus k_{54} \oplus k_{sum}$
Q_2	$\rightarrow$	$Q_{21} = k_{21} \oplus k_{74} \oplus k_{sum}$	$Q_{22} = k_{22} \oplus k_{73} \oplus k_{sum}$	$Q_{23} = k_{32} \oplus k_{62} \oplus k_{sum}$	$Q_{24} = k_{34} \oplus k_{64} \oplus k_{sum}$
Q_3	$\rightarrow$	$Q_{31} = k_{31} \oplus k_{64} \oplus k_{sum}$	$Q_{32} = k_{32} \oplus k_{63} \oplus k_{sum}$	$Q_{33} = k_{22} \oplus k_{72} \oplus k_{sum}$	$Q_{34} = k_{24} \oplus k_{74} \oplus k_{sum}$
Q_4	$\rightarrow$	$Q_{41} = k_{41} \oplus k_{54} \oplus k_{sum}$	$Q_{42} = k_{42} \oplus k_{53} \oplus k_{sum}$	$Q_{43} = k_{12} \oplus k_{82} \oplus k_{sum}$	$Q_{44} = k_{14} \oplus k_{84} \oplus k_{sum}$
Q_5	$\rightarrow$	$Q_{51} = k_{51} \oplus k_{44} \oplus k_{sum}$	$Q_{52} = k_{52} \oplus k_{43} \oplus k_{sum}$	$Q_{53} = k_{41} \oplus k_{51} \oplus k_{sum}$	$Q_{54} = k_{43} \oplus k_{53} \oplus k_{sum}$
Q_6	$\rightarrow$	$Q_{61} = k_{61} \oplus k_{34} \oplus k_{sum}$	$Q_{62} = k_{62} \oplus k_{33} \oplus k_{sum}$	$Q_{63} = k_{31} \oplus k_{61} \oplus k_{sum}$	$Q_{64} = k_{33} \oplus k_{63} \oplus k_{sum}$
Q_7	$\rightarrow$	$Q_{71} = k_{71} \oplus k_{24} \oplus k_{sum}$	$Q_{72} = k_{72} \oplus k_{23} \oplus k_{sum}$	$Q_{73} = k_{21} \oplus k_{71} \oplus k_{sum}$	$Q_{74} = k_{23} \oplus k_{73} \oplus k_{sum}$
Q_8	$\rightarrow$	$Q_{81} = k_{81} \oplus k_{14} \oplus k_{sum}$	$Q_{82} = k_{82} \oplus k_{13} \oplus k_{sum}$	$Q_{83} = k_{11} \oplus k_{81} \oplus k_{sum}$	$Q_{84} = k_{13} \oplus k_{83} \oplus k_{sum}$

Fig. 5 Constructing of Q_i and algebraic transformations of session keys

Table 1 Different combinations of states between two hyper-chaotic systems

Serial number	*R*-channel		*G*-channel		*B*-channel	
	Key_$r1$	Key_$r2$	Key_$g1$	Key_$g2$	Key_$b1$	Key_$b2$
0	x_1	x_2	x_3	x_4	x_5	x_6
1	x_2	x_3	x_4	x_5	x_6	x_7
2	x_3	x_4	x_5	x_6	x_7	x_8
3	x_4	x_5	x_6	x_7	x_8	x_1
4	x_5	x_6	x_7	x_8	x_1	x_2
5	x_6	x_7	x_8	x_1	x_2	x_3
6	x_7	x_8	x_1	x_2	x_3	x_4
7	x_8	x_1	x_2	x_3	x_4	x_5

this value after encryption, then the sum.image variable reached at the encryptor and decryptor, which are not the same and thus the decryption would be impossible.

Step 4 compute the $\bar{x}$ using the following formula:

$$\text{sum.image} = \text{sum.image} - (r(i) + g(i) + b(i)), \tag{10}$$

$$\bar{x} = \text{floor}\left(\text{mod}\left(\frac{\text{sum.image}}{256^5} \times 10^{10}, 8\right)\right). \tag{11}$$

As $\bar{x} \in [0,7]$, we select from Table 1, the corresponding group that can be used to perform encryption operation if $\bar{x}$ equals to the serial number of sequence of the group.

Remark 2 The sum.image variable is used in Eq. (10) for encryption of the ith pixel. Because of the fact that the algorithm needs to be reversible, the ith pixel is not used for the formation of the sum. image.

Remark 3 To increase the effect of the sum.image variables, Eqs. (9) and (11) are used to increase the sensitivity of this algorithm to a change in even one bit of the image. According to Eq. (11), eight different values are given to x in Table 1 which results in selecting six keys in each iteration of the diffusion function. These steps help increase the random nature of our method.

Step 5 calculate the ith pixel of cipher-images (cr_i, cg_i, and cb_i) using the values of the ith pixel of original images (r_i, g_i, and b_i), the previously operated pixels (cr_{i-1}, cg_{i-1}, and cb_{i-1}), and ith chaotic keys corresponding to serial number $\bar{x}$ according to Table 1:

$$\text{cr}_i = \text{bitxor}(r_i, \text{bitxor}(\text{mod}(\text{cr}_{i-1} + \text{key\_}r1, 256)\, \text{key\_}r2)), \tag{12}$$

$$\text{cg}_i = \text{bitxor}(g_i, \text{bitxor}(\text{mod}(\text{cg}_{i-1} + \text{key\_}g1, 256)\, \text{key\_}g2)), \tag{13}$$

$$\text{cb}_i = \text{bitxor}(b_i, \text{bitxor}(\text{mod}(\text{br}_{i-1} + \text{key\_}b1, 256)\, \text{key\_}b2)), \tag{14}$$

where bitxor(x, y) returns the result after a bitwise XOR operation.

Remark 4 In Eq. (12), a hyper-chaotic system is employed to generate key_$r1$ and key_$r2$; therefore, these keys are pseudo-random. On the other hand, since the "add," "mod," and "XOR" operators do not disturb the pseudo-randomness nature of these keys, therefore, according to the equation below, the encrypted image has a random nature [28,29]. Thus, a uniform histogram is reached. Equations (13) and (14) are the same as Eq. (12).

$$\mathrm{cr}_i = \mathrm{bitxor}\Bigg(\underbrace{\Bigg(\underbrace{r_i, \mathrm{bitxor}\Bigg(\underbrace{\mathrm{mod}\Bigg(\underbrace{\mathrm{cr}_{i-1} + \underbrace{\mathrm{key\_}r1}_{\text{Pseudo-random}}}_{\text{Pseudo-random}}, 256\Bigg)}_{\text{Pseudo-random}}, \underbrace{\mathrm{key\_}r2}_{\text{Pseudo-random}}\Bigg)}_{\text{Pseudo-random}}\Bigg)}_{\text{Pseudo-random}}\Bigg) \tag{15}$$

Step 6 set $i \leftarrow i + 1$ and return to step 4 until i reaches L. Then, we get the encrypted arrays $\mathrm{CR} = \{\mathrm{cr}_1, \mathrm{cr}_2, \ldots, \mathrm{cr}_L\}$, $\mathrm{CG} = \{\mathrm{cg}_1, \mathrm{cg}_2, \ldots, \mathrm{cg}_L\}$, and $\mathrm{CB} = \{\mathrm{cb}_1, \mathrm{cb}_2, \ldots, \mathrm{cb}_L\}$. By reshaping these sequences into $M \times N$ images, we obtain the cipher-images. Then, we recombine them into color image C. C is the encrypted image.

Remark 5 In the encryption procedure, from Eq. (7) to Eq. (14), one can see that the pixel gray values of the original image are employed to modify the final encryption keys. Hence, the final encryption keys depend on not only the initial state values of the hyper-chaos systems but also on the original image. When different plain-images are encrypted, the corresponding key stream is not the same. The attacker cannot obtain useful information by encrypting some special images since the resultant information is related to those chosen-images. Therefore, the proposed scheme can well resist the known-plaintext and the chosen-plaintext attacks.

3.3 Decryption algorithm

The decryption procedure is similar to that of the encryption process except that some steps are followed in a reversed order.

Step 1 convert the encrypted color image C into three matrixes CR, CG, and CB.

Step 2 transform each matrix into one-dimensional vectors denoted by $\mathrm{CR} = \{\mathrm{cr}_1, \mathrm{cr}_2, \ldots, \mathrm{cr}_L\}$, $\mathrm{CG} = \{\mathrm{cg}_1, \mathrm{cg}_2, \ldots, \mathrm{cg}_L\}$, and $B = \{\mathrm{cb}_1, \mathrm{cb}_2, \ldots, \mathrm{cb}_L\}$.

Step 3 set $i \leftarrow L$; $\bar{x} \leftarrow 0$; and $sum.image \leftarrow 0$.

Step 4 performing the reverse operations corresponds to the diffusion operation in the encryption algorithm from the last pixel to the first pixel. The equations for removing the effect of diffusion stage are illustrated as follows:

$$r_i = \mathrm{bitxor}\left(\mathrm{cr}_i, \mathrm{bitxor}\left(\mathrm{mod}\left(\mathrm{cr}_{i-1} + \mathrm{key\_}r1, 256\right) \mathrm{key\_}r2\right)\right), \tag{16}$$

$$g_i = \mathrm{bitxor}\left(\mathrm{cg}_i, \mathrm{bitxor}\left(\mathrm{mod}\left(\mathrm{cg}_{i-1} + \mathrm{key\_}g1, 256\right) \mathrm{key\_}g2\right)\right), \tag{17}$$

$$b_i = \mathrm{bitxor}\left(\mathrm{cb}_i, \mathrm{bitxor}\left(\mathrm{mod}\left(\mathrm{br}_{i-1} + \mathrm{key\_}b1, 256\right) \mathrm{key\_}b2\right)\right), \tag{18}$$

$$\mathrm{sum.image} = \mathrm{sum.image} + (r(i) + g(i) + b(i)), \tag{19}$$

$$\bar{x} = \mathrm{floor}\left(\mathrm{mod}\left(\frac{\mathrm{sum.image}}{256^5} \times 10^{10}, 8\right)\right). \tag{20}$$

Step 5 set $i \leftarrow i - 1$; return to step 4 until i reaches 1. To obtain the first pixel of images (r_1, g_1, and b_1), we use the initial value cr_{-1}, cg_{-1}, and cb_{-1} which are calculated using Eq. 9.

By reshaping the sequences $R = \{r_1, r_2, \ldots, r_L\}$, $G = \{g_1, g_2, \ldots, g_L\}$, and $B = \{b_1, b_2, \ldots, b_L\}$ into an $M \times N$ image, we obtain the recovered image.

4 Performance and security analysis

A good encryption algorithm should resist all kinds of known attacks, such as exhaustive attack, statistical attack, and chosen-plaintext/ciphertext attack [28–32]. To demonstrate the security and efficiency of our algorithm, we use six 256×256 color images as the original ones. The external secret key is selected as "1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32" and the integration step $h = 0.001$. The integer N_0 is chosen to be 125. In this section, the proposed image cryptosystem is analyzed using different security measures. These measures consist of key space analysis, statistical analysis (information entropy analysis, Peak Signal-to-Noise Ratio (PSNR), and correlation), and sensitivity analysis [Mean Absolute Error (MAE), plaintext and key sensitivity]. Each of these measures is described in detail in the following subsections.

4.1 Key space analysis

Given our previous explanation, the more complex the dynamic system, the more difficult it is to predict the generated sequences. In today's cryptosystems, the only secret is the "Key" which is the control parameters and/or initial values of the PRNG. This "Key" should be sensitive to its slightest modifications. Thus, an attacker would not be able to guess the "Key" even if he known the algorithm or the dynamic system used. In other words, when dealing with cryptography, a general assumption is made that the architecture of the cryptosystem is public, i.e., known by everyone. What secret is the encryption/decryption key when using private key cryptosystems [28,29].

A good image encryption method should have a large key space in order to resist an exhaustive attack effectively [32–36]. Key space is the total number of different keys that can be used in an encryption system. The size of the key space should be larger than 2^{100} to provide a high level of security [32,33]. Since the secret key of the proposed method is 256-bit long, the key space is about 2^{256}. The key space of the proposed algorithm is compared with other image encryption algorithms in Table 2. The table shows that the key space of our suggested method is large enough to resist any brute-force attack.

4.2 Statistical analysis

An ideal image encryption method should be resistive against any statistical attack [38,39]. In other words, the encrypted images should possess certain random properties. To demonstrate the robustness of our cryptosystem, we have performed statistical analysis by computing the histograms, the PSNR, the entropy, and the correlations for the plain-image and cipher-image. Different standard images have been tested, and we have found that the intensity values meet the requirement.

4.2.1 Histogram

To protect the information of the original image to withstand the statistical attacks, it is essential for the encrypted image to bear no statistical similarity to the original image. The histogram of the encrypted image has to be uniform [40–42]. An opponent finds it difficult to extract the pixels' statistical nature of the plain-image from the cipher-image, and the algorithm can resist a chosen-plaintext or known-plaintext attack. In other words, a hyper-chaotic system, due to its random nature, has a high entropy approximately equal to the number of symbols (8 in the grayscale image). Consequently, hyper-chaotic function has an output equally distributed on the image pixels and causes the uniform generation of pixels in crypto image. This means a uniform histogram and a high entropy.

Table 2 The comparison of key space between our proposed method and other cryptosystem

Algorithm	Key space
Proposed algorithm	$2^{256} = 1.1579 \times 10^{77}$
Ref. [6]	10^{42}
Ref. [8]	2^{128}
Ref. [11]	2^{186}
Ref. [13,14]	$10^{70} \approx 2^{233}$
Ref. [34]	2^{240}
Ref. [37]	10^{56}

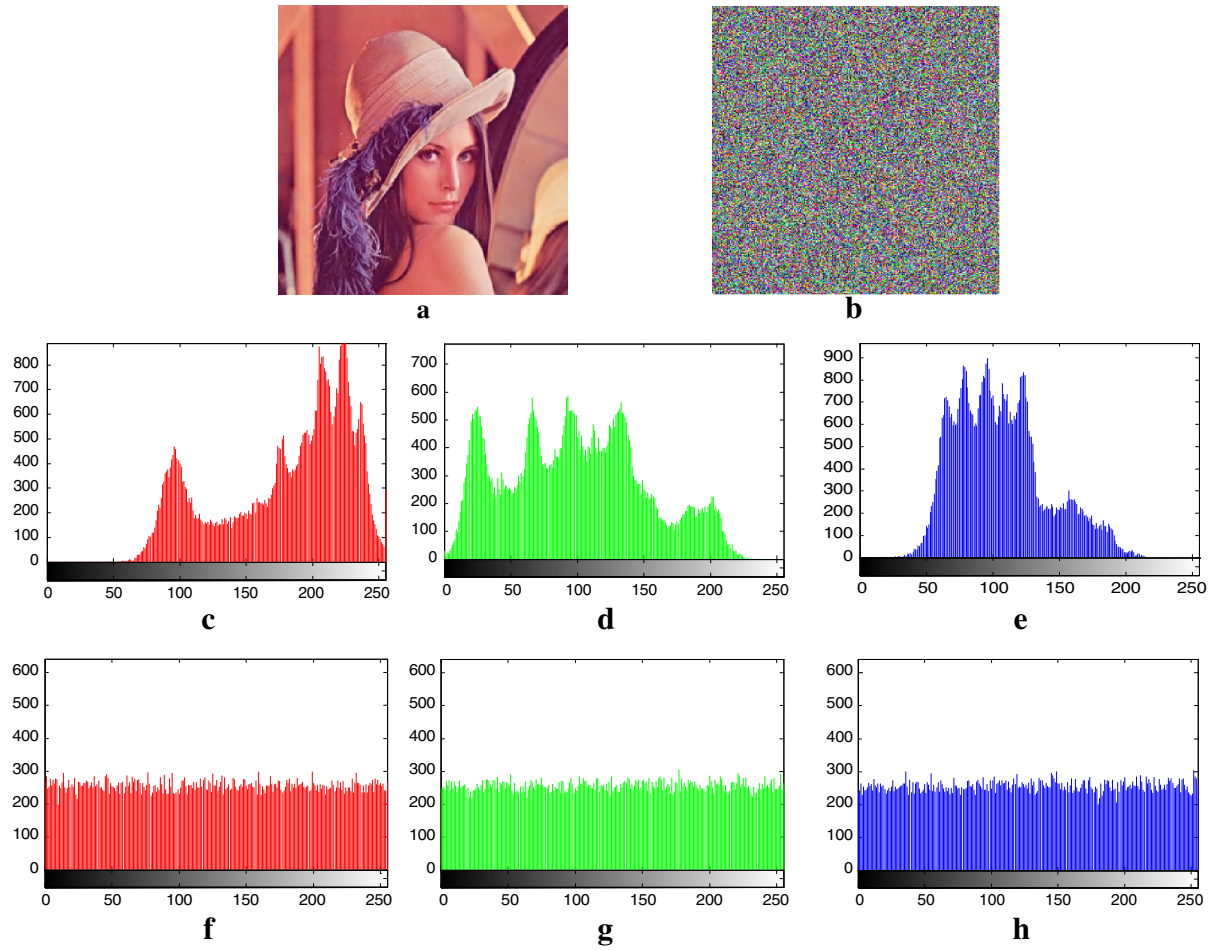

Fig. 6 **a** Lena image (256 × 256); **b** Lena encryption; Histogram of the original image of Lena in the **c** *Red*, **d** *Green*, and **e** *Blue* components; Histogram of the encrypted image of Lena in the **f** *Red*, **g** *Green*, and **h** *Blue* components

One must also pay attention to the fact that many cryptosystems have been cracked [3,5,11,14] regardless of the PRNG system employed. This is due to the weakness of their diffusion function (encryption algorithm), and this is why in addition to a suitable PRNG system, we also need a strongly resistant encryption algorithm (diffusion and confusion algorithm).

In our proposed method, any one of the hyper-chaotic systems may be used to obtain an approximately uniform histogram in the encrypted image. But, according to our previous discussion (see Sect. 2), it is obvious that the more ideal the PRNG system (here a hyper-chaotic system), the more uniform would be the histogram of the cipher-image. By "more ideal," we mean a higher randomness in generated numbers and higher entropy. Thus, the cipher-image has a more uniform histogram and higher entropy leading to lower degree of correlation among pixels and, therefore, a higher quality of the cryptosystem. Also, using the image pixels leads to resistance against differential attacks.

The histograms of several 256 × 256 color images were analyzed with different contents and natures. Test results show that histograms of cipher-images are very uniform and obviously different from the histogram of the plain-images which makes statistical attacks difficult. Figure 6 shows one typical result. The figures show clearly the uniformity and random-like appearance of the histogram of the encrypted image. It demonstrates that our encryption scheme has covered up all the characters of the plain-image and shows good performance of balanced 0–1 ratio and zero correlation.

4.2.2 Peak signal-to-noise ratio

Even though an image histogram is a useful tool, unfortunately, it does not tell us much about the distance between plain and cipher images. A better alternative would be to use similarity measurement metrics such as the popular PSNR. An encrypted image should be significantly different from the original one. As a measure of the level of encryption of an image, we calculate the PSNR. The PSNR should be a low value corresponding to great difference between the original image and the encrypted image. Mathematically, PSNR is defined as

$$\mathrm{PSNR} = 20\log_{10}\left[\frac{255}{\sqrt{\mathrm{MSE}}}\right](db)\,, \tag{21}$$

$$\mathrm{MSE} = \frac{1}{M \times N}\sum_{i=1}^{M}\sum_{j=1}^{N}(a\,(i,\,j) - b\,(i,\,j))^2, \tag{22}$$

where MSE is the Mean Square Error between the original image and the encrypted image; M and N are the width and the height of the test image, respectively; $a(i,\,j)$ and $b(i,\,j)$ are two pixel gray values of the original image and encrypted image at the location $(i,\,j)$, respectively. The larger the MSE value, the better the encryption security. The effectiveness of the proposed method, evaluated in terms of MSE and PSNR for all six images, is tabulated in Table 3.

The average of the PSNR values for the encrypted images is about 8.1717, which is considerably lower than Refs. [11,32,38].

Moreover, to compare the MSE values for Lena encryption, the MSE values of the reported schemes in the recent years are mentioned as follows. The algorithm of Ref. [41] reports MSE = 8,369, the Norouzi's algorithm reports MSE = 9,404 [32], the Singh's algorithm mentioned in Refs. [41] reports MSE = 5,800, the Wang's algorithm mentioned in Ref. [41] reports MSE = 3,300, the Prasanna's algorithm mentioned in Ref. [41] reports MSE = 8,760, and the proposed algorithm reports MSE = 9,790. Compared with these existing results, the MSE results of the proposed algorithm for the cipher-image Lena are better than these algorithms.

4.2.3 Information entropy

Entropy is a measure of uncertainty [28,29,32]. We can use it to express uncertainties in the image information. The entropy can determine the distribution of gray-level values in the image. If the distribution of gray-level values is more uniform, then the information entropy is greater. The higher the value of entropy of an encrypted image, the better is the security. The entropy is given as

$$H\,(s) = \sum_{i=0}^{2^M-1} P\,(s_i)\log_2\frac{1}{P\,(s_i)}, \tag{23}$$

where $P(s_i)$ denotes the probability of symbol s_i and 2^M is the total states of the information source. A truly Random Image (RI) has a uniform distribution of pixel intensities in the interval [0, 255], i.e., $P(\mathrm{RI}_i) = 1/256$ for all $i \in [0, 255]$, hence, $H\,(\mathrm{RI}) = 8$ bits. In other

Table 3 Comparison of MSE and PSNR of different encrypted images of the proposed method

Image	PSNR			MSE		
	Red	Green	Blue	Red	Green	Blue
Lena	7.8160	8.6070	9.6404	10,748	8,962	7,064
Peppers	9.0509	7.6578	7.6594	8,091	11,151	11,147
Couple	6.6851	6.1268	6.0326	13,950	15,863	16,212
Baboon	8.7359	9.2886	8.4064	8,699	7.660	9,385
Home	9.7755	8.7583	8.3067	6,848	8,655	9,603
Tree	8.7150	7.5679	8.2590	8,741	11,384	9,709
Average	8.4631	8.0011	8.0508	9,513	9,337	10,520
Average of three components	8.1717			9,790		
Ref. [11] (average)	9.0348			8,121		
Ref. [38] (average)	9.0486			8,095		

Table 4 The information entropy of plain/cipher images

Image	Original image			Encrypted image		
	Red	Green	Blue	Red	Green	Blue
Lena	7.2763	7.5834	7.0160	7.9972	7.9973	7.9971
Peppers	7.3830	7.6072	7.1250	7.9971	7.9973	7.9967
Couple	6.2499	5.9642	5.9309	7.9975	7.9972	7.9976
Baboon	7.7603	7.4887	7.7787	7.9972	7.9969	7.9972
Home	6.4311	6.5389	6.2320	7.9967	7.9972	7.9971
Tree	7.2104	7.4136	6.9207	7.9973	7.9970	7.9975
Average of six images	7.0518	7.0993	6.8339	**7.9972**	**7.9972**	**7.9972**
Lena	Ref. [13]			7.9971	7.9969	7.9962
Lena	Ref. [42] (bit-level permuted)			7.9791	7.9802	7.9827
Baboon				7.9579	7.9690	7.9633
Lena	Ref. [42] (Usin Chen system)			7.9871	7.9881	7.9878
Baboon				7.9877	7.9881	7.9877

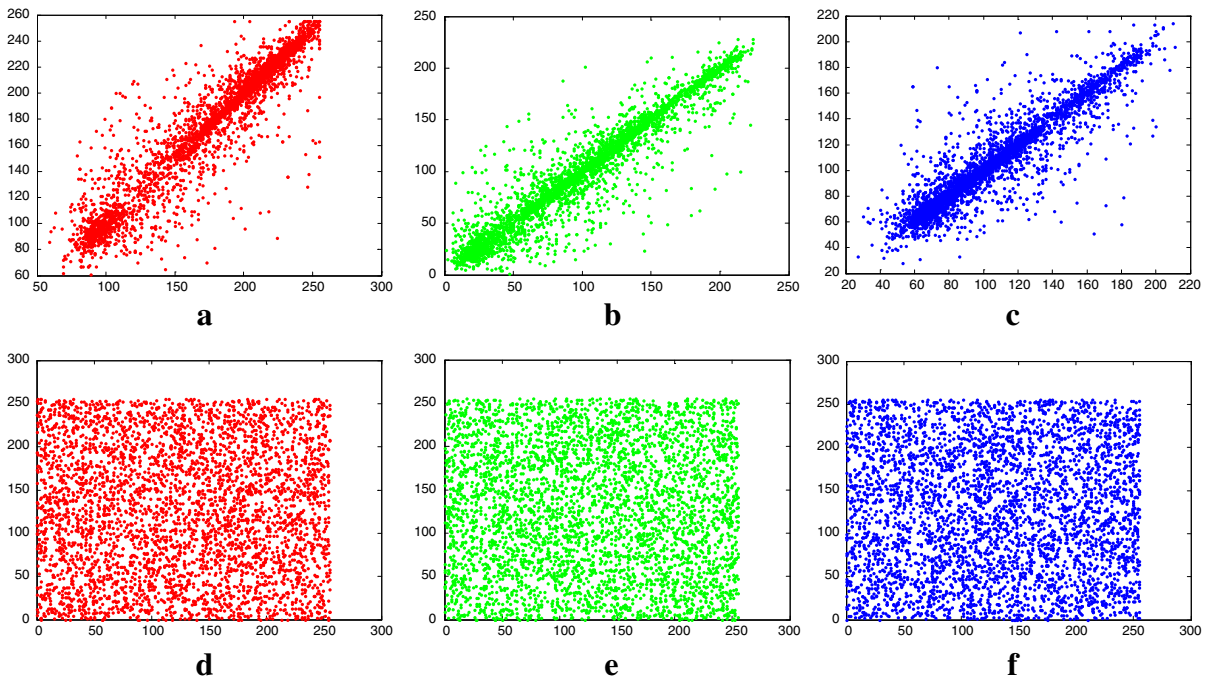

Fig. 7 Correlation analysis of two horizontally adjacent pixels: frames (**a**), (**b**), and (**c**), respectively, show the distribution of two horizontally adjacent pixels in the plain-image of Lena in the **a** *red*, **b** *green*, and **c** *blue* components. Frames (**d**), (**e**), and (**f**), respectively, show the distribution of two horizontally adjacent pixels in the encrypted image of Lena in the **a** *red*, **b** *green*, and **c** *blue*, components; obtained using the proposed scheme

words, for an ideally random image, the value of the information entropy is 8.

We have computed the information entropy for six standard plain-images and their corresponding cipher-images. The results are shown in Table 4. Also, the entropy averages of six images are calculated and shown in the ninth row of this Table. The values obtained are very close to the theoretical value of

Table 5 Correlation coefficients of two adjacent pixels in the plain/cipher image of LENA

Algorithm	Lena	Component	Orientation		
			Horizontal	Vertical	Diagonal
Proposed algorithm	Original image	Red	0.9436381	0.9428391	0.8947478
		Green	0.9747134	0.9733132	0.9474552
		Blue	0.9283210	0.9272441	0.8720709
		Average	0.948891	0.947799	0.904758
	Encrypted image	Red	−0.0002687	−0.0006838	0.0002470
		Green	−0.0006084	−0.0006938	−0.0001424
		Blue	−0.0007598	−0.0001638	−0.000188
		Average of absolute values	0.000546	0.000514	0. 000192
Ref. [13]	Encrypted image	Red	0.0054	0.0059	0.0013
		Green	0.0062	0.0016	0.0022
		Blue	0.0017	0.0029	0.0026
		Average of absolute values	0.0044	0.0034	0.0020
Ref. [39]	Encrypted image	Red	0.0017	0.0021	0.0048
		Green	0.0019	0.0045	0.0027
		Blue	0.0042	0.0051	0.0036
		Average of absolute values	0.0026	0.0039	0.0037

$H = 8$. This means that the cipher-image is close to a random source, and information leakage in the encryption process is negligible. Comparing it with the other existing algorithms, such as [13,42], the proposed algorithm is much closer to the ideal situation. So, our output acts similar to a random one and outperforms other algorithms. In other words, the new encryption system is secure against the entropy attack.

4.2.4 Correlations of two adjacent pixels

Each pixel in an image is highly correlated with its adjacent pixels either in horizontal, vertical, or diagonal direction. This can be utilized to carry out cryptanalysis. Therefore, a secure encryption method should remove the correlation between adjacent image pixels in order to improve the resistance against statistical analysis. In an ideal condition, there should not be any relation between the pixels of the encrypted image [correlation coefficient (CC) ≈0]. To test the correlation between two adjacent pixels in a ciphered image, the following procedure was carried out [32]. First, we randomly select 4,096 pairs of two adjacent pixels from an image. Then, we calculate the CC of each pair using the following formulas:

$$r_{xy} = \operatorname{cov}(x, y)/\sqrt{D(x)D(y)},$$

$$E(x) = \frac{1}{N}\sum_{i=1}^{N} x_i,$$

$$D(x) = \frac{1}{N}\sum_{i=1}^{N} (x_i - E(x))^2, \tag{24}$$

$$\operatorname{cov}(x, y) = \frac{1}{N}\sum_{i=1}^{N} (x_i - E(x))(y_i - E(y)),$$

where x and y are the gray values of two adjacent pixels in the image. Figure 7 shows the correlation distribution of two vertically adjacent pixels in the original image and that in the ciphered image. According to Eq. (24), the CC of the plain-image of Lena is 0.948891and its ciphered image is 0.000546 along the horizontal direction. Similar results for vertical and diagonal directions are also obtained and are shown in Table 5.

Both Table 5 and Fig. 7 show significant reduction in relevance of adjacent pixels. This indicates that our proposed algorithm has effectively removed the correlation of adjacent pixels in the plain-image, so that neighboring pixels in the cipher-image virtually have no correlation. Compared with the algorithms proposed by Refs. [13,39], it shows superior performance.

Table 8 compares the $\mathrm{NPCR}_{\mathrm{R,G,B}}$ and $\mathrm{UACI}_{\mathrm{R,G,B}}$ for our proposed scheme, Zhi-liang Zhu's scheme [6],

Table 6 The results of CC values between original and encrypted images

Image	CC		
	Red	Green	Blue
Lena	−0.0031	0.0043	−0.0025
Peppers	−0.0047	−0.0024	−0.0028
Couple	0.0054	0.0003	−0.0058
Baboon	0.0059	−0.0023	−0.0037
Home	−0.0063	−0.0012	−0.0020
Tree	0.0021	−0.0071	−0.0019
Average of absolute values	0.0046	0.0029	0.0031
Average ($CC_{R,G,B}$)	0035		
Ref. [11]	0.003673		

Yong Wang's scheme [10], Congxu Zhu's scheme [11], Liu Jin-mei's scheme [12], Xiaopeng Wei's scheme [13], Mohammad Seyedzadeh's scheme [33], Sahar Mazloom's scheme [34], Behnia scheme [35], Sun scheme [36], and Xiaoling Huang's scheme [37] on "Lena."

4.2.5 Correlations between plain and cipher-images

The correlations between various pairs of plain/cipher images have been analyzed by computing the 2D CCs between input and encrypted images [11,32]. The CC is calculated as follows:

$$\mathrm{CC} = \frac{\sum_{i=1}^{M}\sum_{j=1}^{N}\left(A_{ij}-\bar{A}\right)\left(B_{ij}-\bar{B}\right)}{\sqrt{\left(\sum_{i=1}^{M}\sum_{j=1}^{N}\left(A_{ij}-\bar{A}\right)^{2}\right)\left(\sum_{i=1}^{M}\sum_{j=1}^{N}\left(B_{ij}-\bar{B}\right)^{2}\right)}}, \tag{25}$$

where A and B represent the plain-image and cipher-image, respectively; $\bar{A}$ and $\bar{B}$ are the mean values of the elements of matrices A and B, respectively; and M and N are the height and width of the plain/cipher image, respectively.

In Table 6, the CC of the encryption is pointed out. It is clear that the CCs between various pairs of the original and encrypted images are very small (or practically zero). Considering CC values, our proposed algorithm also shows better performance in comparison to Ref. [11]. Therefore, the encrypted and original images are significantly different.

4.3 Differential analysis

As a general requirement for all image encryption algorithms, the cipher-image should be greatly different from its original form. The meaningful relationship between an original image and its encrypted image can be revealed using a slightly modified cipher-image. An attacker can crack the cipher by tracing the difference of the two cipher-images. This kind of attack is called differential attack. The secure image encryption algorithm has to be highly sensitive to the plaintext and the key, so that any changes in the plaintext or the key should affect the whole ciphertext.

4.3.1 Plaintext sensitivity

The image encryption algorithm should be sensitive to a tiny change in the original image, which means that one-pixel change leads to a completely different cipher-image. Two criteria are generally used to measure the influence of a one-pixel change in plain-image on the corresponding cipher-image. They are the number of pixels change rate (NPCR) and the unified aver-

Table 7 Plaintext sensitivity

Image	NPCR (average of 50 iterations for each component)			UACI (average of 50 iterations for each component)		
	Red	Green	Blue	Red	Green	Blue
Lena	99.8038	99.7942	99.8114	33.8228	33.4471	33.6011
Peppers	99.8192	99.7996	99.8015	33.4707	33.4826	33.7332
Couple	99.8050	99.7987	99.7814	33.5340	33.7524	33.6882
Baboon	99.8031	99.8019	99.8071	33.5079	33.4228	33.4281
Home	99.8066	99.8118	99.8099	33.5422	33.6067	33.4961
Tree	99.7943	99.7966	99.8044	33.4777	33.4450	33.5400
Average (for each component)	99.8053	99.8005	99.8026	33.5592	33.5261	33.5811
Avarage (900 iterations)	99.8028			33.5555		

Table 8 Comparison of $\mathrm{NPCR}_{\mathrm{R,G,B}}$ and $\mathrm{UACI}_{\mathrm{R,G,B}}$ on 'Lena'

Algorithm	Average ($\mathrm{NPCR}_{\mathrm{R,G,B}}$) (%)	Average ($\mathrm{NPCR}_{\mathrm{R,G,B}}$) (%)
Proposed algorithm	99.8031	33.6237
Ref. [6]	99.6052	33.4000
Ref. [10]	99.6110	33.4630
Ref. [11]	99.6100	33.4700
Ref. [12]	99.6000	33.5000
Ref. [13]	99.2172	33.4055
Ref. [33]	99.6828	33.4898
Ref. [34]	99.6410	33.3620
Ref. [35]	0.46	0.39
Ref. [36]	0.40	0.3192
Ref. [37]	99.3097	33.4553

age changing intensity (UACI) which are calculated by

$$\mathrm{NPCR}_{\mathrm{R,G,B}} = \frac{\sum_{i,j} D_{\mathrm{R,G,B}}(i,j)}{M \times N} \times 100\ \%, \tag{26}$$

$$\mathrm{UACI}_{\mathrm{R,G,B}} = \frac{1}{M \times N} \times \left[\sum_{i,j} \frac{\left| C_{\mathrm{R,G,B}}(i,j) - C'_{\mathrm{R,G,B}}(i,j) \right|}{255} \right] \times 100\ \%, \tag{27}$$

where C and C' are the two cipher-images whose corresponding plain-images have only one-pixel difference. The grayscale values of the pixels at position (i, j) of C and C' are denoted as $C(i, j)$ and $C'(i, j)$, respectively. The difference array $D_{\mathrm{R,G,B}}(i, j)$ is determined by the following rule: If $C_{\mathrm{R,G,B}}(i, j) = C'_{\mathrm{R,G,B}}(i, j)$, then $D_{\mathrm{R,G,B}}(i, j) = 0$; otherwise, $D_{\mathrm{R,G,B}}(i, j) = 1$.

To evaluate plaintext sensitivity of our proposed scheme, the following procedure was carried out:

- Plain-image is encrypted first.
- Then, one pixel in the original image is randomly selected and changed. The modified image is encrypted again using the same key.
- Finally, the two cipher-images are compared pixel-by-pixel (the NPCR and UACI values are computed).

This kind of test is performed over 150 times with different images (900 times for all images). Table 7 provides the data related to the experimental results obtained by our proposed algorithm. As can be seen from the computer simulation results, the proposed cryptosystem only needs a minimum of one round to achieve a high performance such as NPCR >99.80 % and UACI >33.56 %. Therefore, the proposed algorithm can resist the differential attack if the number of rounds is at least one.

As it is obvious from the simulation results, the proposed cryptosystem achieves high performance by having $\mathrm{NPCR}_{\mathrm{R,G,B}} > 99.8028$ and $\mathrm{UACI}_{\mathrm{R,G,B}} > 33.5555$, and can well resist the known- plaintext and the chosen-plaintext attacks.

4.3.2 Key sensitivity analysis

Extreme key sensitivity is an important feature for any good image encryption cryptosystem which guarantees the security of the algorithm against the brute-force attack to some extent. Indeed, a small deviation in the input should cause a large change in the output. The hyper-chaos systems are sensitive to initial values which ensure the key sensitivity of the encryption scheme. Two key sensitivity tests were performed.

4.3.2.1 First key sensitivity test The encrypted image generated by the image encryption algorithm should be very sensitive to the secret external key; i.e., if two slightly different keys are used to encrypt the same plain image, then the two cipher-images produced should be completely independent of each other.

- A 256 × 256 color image is encrypted using "1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32" as the first set of keys or KEY1.

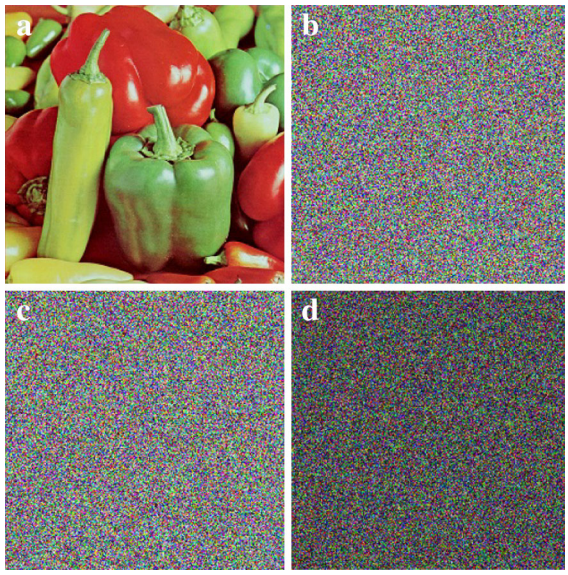

Fig. 8 Key sensitivity test: **a** peppers image; **b** encrypted image with KEY1; **c** encrypted image with KEY2: "0, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32"; and **d** Difference between **b** and **c**

- Then, one bit in the external secret key is toggled and used to encrypt the same plain-image.
- The two cipher-images are compared pixel-by-pixel. Figure 8 shows the test result.

This kind of test is performed over 60 times on different images (360 times for all images). The results shown in Table 9 are obtained by simulation. It is clear that NPCR is over 99.61 % and UACI is over 33.4 %. This means that more than 99.61 % of all pixels in cipher-image change their gray-level values when only one bit of the key is toggled. The results can illuminate that our suggested image encryption algorithm is very sensitive with respect to tiny changes in the external secret key. In other words, it has strong ability of resisting differential attacks.

4.3.2.2 Second key sensitivity test The encrypted image cannot be decrypted correctly although there is a trivial difference between the encryption and decryption keys. Figure 9 illustrates the sensitivity of our scheme to the external secret key. Figure 9c is the decrypted image with the same external secret key (KEY1) as that used in encryption algorithms. Figure 9d is the decrypted image with all the bits of the key being the same as that used in encryption algorithm except the last bit.

So, it can be concluded that our hyper-chaos encryption algorithm is sensitive to the external secret key; a small change (e.g., modifying only one bit) of the external secret key will generate a completely different decryption result and cannot yield the correct plain-image.

4.3.3 Mean absolute error

The performance of the difference between the cipher- and plain-images is measured by the MAE. Let $C(i, j)$ and $P(i, j)$ be the gray levels of the pixels at the ith row and jth column of a $M \times N$ cipher and plain-image, respectively. The MAE between these two images is defined as

$$\mathrm{MAE} = \frac{1}{M \times N} \sum_{j=1}^{M} \sum_{i=1}^{N} |C(i, j) - P(i, j)|. \qquad (28)$$

Table 9 Key sensitivity

Image	NPCR (average of 20 iterations for each component)			UACI (average of 20 iterations for each component)		
	Red	Green	Blue	Red	Green	Blue
Lena	99.6126	99.6091	99.6073	33.4979	33.4803	33.4177
Peppers	99.6049	99.6177	99.6046	33.5267	33.4286	33.4464
Couple	99.6119	99.6136	99.6175	33.4518	33.4373	33.4259
Baboon	99.6084	99.6136	99.6040	33.4674	33.4975	33.4081
Home	99.6087	99.6084	99.6107	33.3943	33.5152	33.4395
Tree	99.6101	99.6111	99.6090	33.4837	33.5050	33.4484
Average (for each component)	99.6094	99.6123	99.6089	33.4703	33.4773	33.4310
Avarage (360 iterations)	99.6102			33.4595		

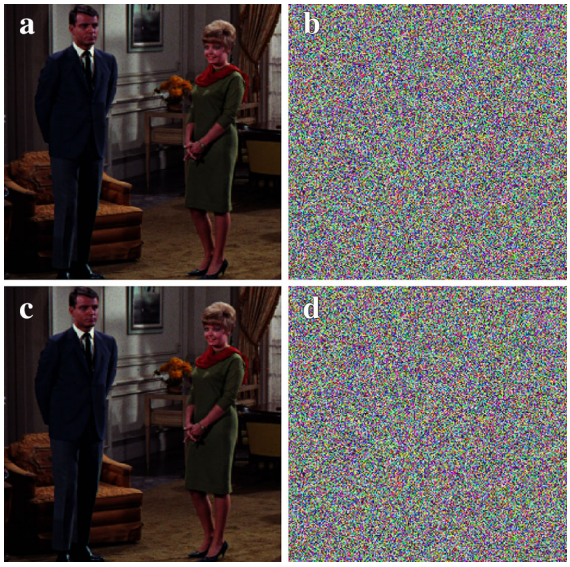

Fig. 9 Key sensitivity test: **a** couple image, **b** encrypted image with KEY 1, **c** decrypted image with KEY 1, and **d** decrypted image with KEY 2

Table 10 The results for the MAE values

Image	MAE		
	Red	Green	Blue
Lena	84.7177	77.5301	75.2204
Peppers	74.3792	86.1866	86.1460
Couple	97.1340	104.5973	105.8835
Baboon	76.6660	72.6091	79.3484
Home	69.4347	76.4666	80.2435
Tree	76.7878	87.2157	80.5876
Average	79.8532	84.1009	84.5716
Average ($CC_{R,G,B}$)	82.8419		
Ref. [32]	80.2		

The larger the MAE value, the better the encryption security. The results for the MAE values are shown in Table 10.

4.4 Speed performance

Regardless of the security considerations, some other aspects on image encryption are also important, such as the running speed for the real-time internet applications. According Table 8, the NPCR and UACI measurements reported in Refs. [35,36] are 0.46, 0.40 and 0.39, 0.3192 %, respectively. They can hardly satisfy any high-performance requirement. On the contrary, it is demonstrated that the proposed method requires only one encryption round (one round of diffusion) to achieve the satisfactory performance with NPCR >99.80 % and UACI >33.55 %.

Table 11 The numbers of permutation and diffusion to achieve NPCR >99.6 % and UACI >33.4 %

Algorithm	Number of permutation	Number of diffusion
New scheme	0	1
Norouzi's [32]	0	2
Wang's [43]	2	2
Wong's [44]	4	2
Lian's [45]	18	6
Xiang's [46]	3	3

Table 12 Encryption speed of the proposed encryption scheme and some other schemes

Algorithm	Speed (Mbit/s)
Proposed algorithm	47.7653
Seyedzadeh [33]	44.9389
Mazloom [34]	9.2115
Ref. [47]	0.3944
Ref. [48]	15.3475
Ref. [49]	8.6573

To make a comparison with other algorithms, the number of permutation and diffusion rounds required to achieve this performance is listed in Table 11. The results show that the number of permutation and diffusion required by the new scheme is fewer than that by other comparable algorithms. Thus, the proposed algorithm indeed leads to a faster encryption speed.

We have also analyzed the speed of the proposed image encryption/decryption technique on a personal computer (PC) with a CPU 2.4 GHz, 4 GB Memory, and 640 GB hard-disk capacity, and the operating system is Microsoft Windows 7 using MATLAB 7.6.0.324 (R2008a) and Eclipse 3.5 compiler. Table 12 shows that the proposed algorithm is very fast compared to the other schemes such as Refs. [33,34,47–49].

The algorithm's speed on a DSP for the color image of size $n \times n$, we compute the number of mathematical operations given in Table 13. According to the proposed results, the total number of used operations is

Table 13 The number of operations used in each of the sections

Section name	Type of operator				
	Sum	Multiplication	XOR	Mod	Floor
Generation the initial conditions (Sect. 3.1.1)	$13n^2 + 13N_0$	$22n^2 + 14N_0$	$4n^2 + 64$	$8n^2$	$8n^2$
Encryption algorithm (Sect. 3.2)	$9n^2$	$2n^2$	$6n^2$	$4n^2$	n^2
Summation of operators	$22n^2 + 13N_0$	$24n^2 + 14N_0$	$10n^2 + 64$	$12n^2$	$9n^2$

$77n^2 + 27N_0 + 64 \approx 77n^2$, and the proposed algorithm has the computational complexity of O (n^2). Supposing that the "add/subtract", "XOR", "mod", and "floor" operators take one cycle and "multiplication/division" operators take two cycles, we can compute the estimated time of hardware execution by multiplying these operators by their related clock cycles and then dividing by the CPU or DSP clock. Therefore, the encryption time on a ADSP-BF5xx DSP with a 750 MHz clock frequency would be less than 9 ms.

It is obvious that running this algorithm on a newer series of DSP with the ability to perform add/multiply operations in one clock would greatly improve the speed.

4.5 Decryption quality

The decryption quality between original and decrypted images can be observed by means of the CC, MSE, and PSNR. For a good decryption, the CC should be near 1 or equal to 1, the MSE should be a small value, and the PSNR should be large. To determine the decryption quality, the decryption algorithm is applied to the encryption of all eight images. The CC, the MSE, and the PSNR are computed for each decrypted image. For the proposed decryption, the CC of each decryption is 1. The MSE of each decryption is 0, and the PSNR of each decryption is infinite. The results suggest that every decryption is accurate. In other words, each decrypted image is identical to the corresponding original image. Thus, a good quality is demonstrated in the proposed decryption.

4.6 Randomness tests for the cipher

For the security of a cryptosystem, the cipher must have some properties such as good distribution, long period, high complexity, and efficiency [50–57]. Recently, the NIST designed a set of different statistical tests to justify randomness of binary sequences produced by either hardware- or software-based cryptographic random or pseudo-random number generators [40].

In order to satisfy these requirements, we have utilized NIST SP 800-22 tests [54], DIEHARD [55], and ENT test suite for testing the randomness of the cipher. Some of these tests consist of a number of subsets. To carry on these tests, we have used 120 sequences of ciphers with the length of 1,000,000 bits. The encrypted image is Lena 24 bits image. To test the cipher randomness, a lot of initial keys are used. The results of the tests are shown in Tables 14, 15, and 16. By analyzing

Table 14 Results of the SP800-22 tests suite for cipher Lena image

Test name	*P* value	Result
Frequency	0.875539	Success
Runs ($M = 10{,}000$)	0.299251	Success
Block-frequency	0.900104	Success
Long runs of ones	0.437274	Success
Rank	0.028181	Success
Spectral DFT	0.689019	Success
No overlapping templates	0.350485	Success
Overlapping templates	0.422034	Success
Universal	0.350485	Success
Lempel ziv complexity	0.170294	Success
Linear complexity	0.739918	Success
Approximate entropy	0.941144	Success
Serial (1)	0.500934	Success
Serial (2)	0.739918	Success
Cumulative sums (1)	0.122325	Success
Cumulative sums (2)	0.517442	Success
Random excursions	0.531816	Success
Random excursions variant	0.430824	Success

Table 15 DIEHARD tests suite for the Lena image

Test name	P value	Result
Birthday spacing	0.309305	Success
Overlapping permutation	0.913173	Success
Binary rank 31 × 31	0.371207	Success
Binary rank 6 × 8	0.890111	Success
Bitstream	0.178442	Success
OPSO	0.663982	Success
OQSO	0.482065	Success
DNA	0.579123	Success
Count the ones 01	0.530422	Success
Count the ones 02	0.822360	Success
Parking lot	0.259096	Success
Minimum distance	0.824483	Success
3DS spheres	0.029493	Success
Squeeze	0.631710	Success
Overlapping sum	0.689073	Success
Runs	0.824604	Success
CrapsRa	0.499557	Success

Table 16 Max grade of ENT test suite for the Lena image

Test name	Average value	Result
Entropy	7.999986	Success
Arithmetic mean	127.5378	Success
Monte Carlo	3.1401497	Success
Chi square	244.94	Success
SCC	0.000313	Success

these results, it can be deduced that our proposed image encryption algorithm successfully passes the NIST SP 800-22, DIEHARD, and ENT tests. Therefore, based on the achieved results, the generated ciphers in our cryptosystem can be claimed, which are quite stochastic.

5 Conclusion

In this paper, we design a color image encryption scheme based on hyper-chaos systems, which consists of two processes; key stream generation process and one-round diffusion process. The first process generates pseudo-random key streams for encryption of the color image based on the two hyper-chaotic systems. A 256 bit-long external secret key is used to generate the initial conditions of two hyper-chaotic systems by applying some algebraic transformations to the key. Our algorithm is based on the final secret key stream which is not only related to the original chaotic key stream but also to the plain-image. The second process employs the image data in order to modify the pixel gray-level values and crack the strong correlations between adjacent pixels of an image simultaneously. In this process, the states which are the combinations of two hyper-chaotic systems are selected according to image data itself and are used to encrypt the red, green, and blue components, respectively.

All parts of the encryption system are simulated using MATLAB and Eclipse 3.5 compiler. Statistical analysis shows that the scheme can well protect the image against statistical attacks. Also, the measured encryption quality shows that the proposed algorithm has a better encryption quality than others. Moreover, the scheme possesses high key sensitivity and has a good ability to resist differential attacks. MAE, NPCR, and UACI were used as three criterions to examine the performance of resistance against differential attacks. The results demonstrate that a swift change in the original image and the key will result in a significant change in the ciphered image and cannot yield the correct plain-image. The decryption procedure is similar to that of the encryption but in the reversed order. Overall, it seems that the proposed algorithm can be a good candidate for image encryption.

Acknowledgments The authors would like to thank the Editor, the anonymous Referees, and Miss Shirin Saberian for their valuable comments and suggestions to improve this paper.

References

1. Zhou, N., Wang, Y., Gong, L., Chen, X., Yang, Y.: Single-channel color image encryption based on iterative fractional fourier transform and chaos. Opt. Laser Technol. **48**, 117–127 (2013)
2. Huang, J.-J., Hwang, H.-E., Chen, C.-Y., Chen, C.-M.: Optical multiple-image encryption based on phase encoding algorithm in the Fresnel transform domain. Opt. Laser Technol. **44**(7), 2238–2244 (2012)
3. Mirzaei, O., Yaghoobi, M., Irani, H.: A new image encryption method: parallel sub-image encryption with hyper chaos. Nonlinear Dyn **67**(1), 557–566 (2012)
4. Zhang, G., Liu, Q.: A novel image encryption method based on total shuffling scheme. Opt. Commun. **284**, 2775–2780 (2011)

5. Belkhouche, F., Qidwai, U.: Binary image encoding using one-dimensional chaotic map. In: Proceedings of the IEEE Annual Techical Conference, pp. 39–43 (2003)
6. Zhu, Z., Zhang, W., Wong, K., Yu, H.: A chaos-based symmetric image encryption scheme using a bit-level permutation. Inf. Sci. **181**, 1171–1186 (2011)
7. Sui, L., Gao, B.: Color image encryption based on Gyrator transform and Arnold transform. Opt. Laser Technol. **48**, 530–538 (2013)
8. Chen, G., Mao, Y., Chui, C.: A symmetric image encryption scheme based on 3D chaotic cat maps. Chaos Solitons Fractals **21**(3), 749–761 (2004)
9. Wang, K., Pei, W., Zou, L., Song, A., He, Z.: On the security of 3D cat map based symmetric image encryption scheme. Phys. Lett. A **343**(6), 432–439 (2005)
10. Wang, Y., Wong, K., Liao, X., Chen, G.: A new chaos-based fast image encryption algorithm. Appl. Soft Comput. **11**(1), 514–522 (2011)
11. Zhu, C.: A novel image encryption scheme based on improved hyperchaotic sequences. Opt. Commun. **285**(1), 29–37 (2012)
12. Liu, J.M., Qiu, S.S., Xiang, F., Xiao, H.J.: A cryptosystem based on multi-chaotic maps. In: International Symposiums on Information Processing, pp. 740–743 (2008)
13. Wei, X., Guo, L., Zhang, Q., Zhang, J., Lian, S.: A novel color image encryption algorithm based on DNA sequence operation and hyper-chaotic system. J. Syst. Softw. **85**(2), 290–299 (2012)
14. Gao, T., Chen, Z.: A new image encryption algorithm based on hyper-chaos. Phys. Lett. A **372**(4), 394–400 (2008)
15. Rhouma, R., Belghith, S.: Cryptanalysis of a new image encryption algorithm based on hyper-chaos. Phys. Lett. A **372**(38), 5973–5978 (2008)
16. Ge, X., Liu, F., Lu, B., Yang, C.: Improvement of Rhouma's attacks on Gao algorithm. Phys. Lett. A **374**(11–12), 1362–1367 (2010)
17. DSouza, R.M., Bar-Yam, Y., Kardar, M.: Sensitivity of ballistic deposition to pseudorandom number generators. Phys. Rev. E **57**, 5044–5052 (1998)
18. Vattulainen, I., Ala-Nissila, T., Kankaala, K.: Physical models as tests of randomness. Phys. Rev. E **52**, 3205–3214 (1995)
19. Behnia, S., Akhavan, A., Akhshani, A., Samsudin, A.: A novel dynamic model of pseudo random number generator. J. Comput. Appl. Math. **235**, 3455–3463 (2011)
20. Lee, P.-H., Chen, Y., Pei, S.-C., Chen, Y.-Y.: Evidence of the correlation between positive Lyapunov exponents and good chaotic random number sequences. Comput. Phys. Commun. **160**, 187–203 (2004)
21. Falcioni, M., Palatella, L., Pigolotti, S.: Properties making a chaotic system a good pseudo random number generator. Phys. Rev. E **72**, 016220 (2005)
22. Gonzlez, C.M., Larrondo, H.A., Rosso, O.A.: Statistical complexity measure of pseudorandom bit generators. Physica A **354**, 281–300 (2005)
23. Choudhury, S.R., Gorder, R.A.V.: Competitive modes as reliable predictors of chaos versus hyperchaos and as geometric mappings accurately delimiting attractors. Nonlinear Dyn. **69**(4), 2255–2267 (2012)
24. Qi, G., Chen, G., Du, S., Chen, Z., Yuan, Z.: Analysis of a new chaotic system. Physica A **352**, 295–308 (2005)
25. Chen, A., Lu, J., Lu, J., Yu, S.: Generating hyperchaotic lu attractor via state feedback control. Physica A **364**, 103–110 (2006)
26. Yujun, N., Xingyuan, W., Mingjun, W., Huaguang, Z.: A new hyperchaotic system and its circuit implementation. Commun. Nonlinear Sci. Numer. Simul. **15**, 3518–3524 (2010)
27. Gao, T., Chen, Z., Yuan, Z., Chen, G.: A hyperchaos generated from Chen's system. Int. J. Mod. Phys. C **17**(4), 471–478 (2006)
28. Menezes, A.J., van Oorschot, P.C., Vanstone, S.A.: Handbook of Applied Cryptography. CRC Press, Boca Raton (1997)
29. Goldreich, O.: Foundations of Cryptography. Weizmann Institute of Science, Rehovot (1995). (fragment of a book)
30. Zhang, Y., Xiao, D., Wen, W., Li, M.: Cryptanalyzing a novel image cipher based on mixed transformed logistic maps. Multimed. Tools Appl. (2013). doi:10.1007/s11042-013-1684-5
31. Ye, G.: A block image encryption algorithm based on wave transmission and chaotic systems. Nonlinear Dyn. (2013). doi:10.1007/s11071-013-1074-6
32. Norouzi, B., Seyedzadeh, S.M., Mirzakuchaki, S., Mosavi, M.R.: A novel image encryption based on hash function with only two-round diffusion process. Multimed. Syst. (2013). doi:10.1007/s00530-013-0314-4
33. Seyedzadeh, S.M., Mirzakuchaki, S.: A fast color image encryption algorithm based on coupled two-dimensional piecewise chaotic map. Signal Process. **92**, 1202–1215 (2012)
34. Mazloom, S., Eftekhari-Moghadam, A.M.: Color image encryption based on coupled nonlinear chaotic map. Chaos Solitons Fractals **42**(3), 1745–1754 (2009)
35. Behnia, S., Akhshani, A., Ahadpour, S., Mahmodi, H., Akhavan, A.: A fast chaotic encryption scheme based on piecewise nonlinear chaotic maps. Phys. Lett. A **366**(4–5), 366, 391–396 (2007)
36. Sun, F., Liu, S., Li, Z., Lü, Z.: A novel image encryption scheme based on spatial chaos map. Chaos Solitons Fractals **38**(3), 631–640 (2008)
37. Huang, X.: Image encryption algorithm using chaotic Chebyshev generator. Nonlinear Dyn. **67**, 2411–2417 (2012)
38. Taneja, N., Raman, B., Gupta, I.: Combinational domain encryption for still visual data. Multimed. Tool Appl. **59**, 775–793 (2012)
39. Shatheesh Sam, I., Devaraj, P., Bhuvaneswaran, R.S.: An intertwining chaotic maps based image encryption scheme. Nonlinear Dyn. **69**, 1995–2007 (2012)
40. Kanso, A., Ghebleh, M.: A novel image encryption algorithm based on a 3D chaotic map. Commun. Nonlinear Sci. Numer. Simul. **17**, 2943–2959 (2012)
41. Borujeni, S.E., Eshghi, M.: Chaotic image encryption system using phase-magnitude transformation and pixel substitution. J. Telecommun. Syst. (2011). doi:10.1007/s11235-011-9458-8
42. Liu, H., Wang, X.: Color image encryption using spatial bit-level permutation and high-dimension chaotic system. Opt. Commun. **284**, 3895–3903 (2011)
43. Wang, Y., Wong, K.W., Liao, X., Xiang, T.: A chaos-based image encryption algorithm with variable control parameters. Chaos Solitons Fractals **41**, 1773–1783 (2009)

44. Wong, K.W., Kwok, B.S., Law, W.S.: A fast image encryption scheme based on chaotic standard map. Phys. Lett. A **372**(15), 2645–2652 (2008)
45. Lian, S., Sun, J., Wang, Z.: A block cipher based on a suitable use of the chaotic standard map. Chaos Solitons Fractals **26**(1), 117–129 (2005)
46. Xiang, T., Liao, X., Tang, G., Chen, Y., Wong, K.W.: A novel block cryptosystem based on iterating a chaotic map. J. Phys. Lett. A **349**, 109–115 (2006)
47. Liu, Z.X.S., Sun, J.: An improved image encryption algorithm based on chaotic system. J. Comput. **4**, 1091–1100 (2009)
48. Akhshani, A., Akhavan, A., Lim, S.-C., Hassan, Z.: An image encryption scheme based on quantum logistic map. Commun. Nonlinear Sci. Numer. Simul. **17**, 4653–4661 (2012)
49. Wang, X., Teng, L., Qin, X.: A novel colour image encryption algorithm based on chaos. Signal Process. **92**, 1101–1108 (2012)
50. Wang, X., Liu, L.: Cryptanalysis of a parallel sub-image encryption method with high-dimensional chaos. Nonlinear Dyn. **73**(1–2), 795–800 (2013)
51. Li, C., Zhang, L.Y., Ou, R., Wong, K.W., Shu, S.: Breaking a novel colour image encryption algorithm based on chaos. Nonlinear Dyn. **70**(4), 2383–2388 (2012)
52. Zhang, Y., Li, C., Li, Q., Zhang, D., Shu, S.: Breaking a chaotic image encryption algorithm based on perceptron model. Nonlinear Dyn. **69**(3), 1091–1096 (2012)
53. Li, C., Liu, Y., Xie, T., Chen, M.Z.Q.: Breaking a novel image encryption scheme based on improved hyperchaotic sequences. Nonlinear Dyn. **73**(3), 2083–2089 (2013)
54. Rukhin, A., et al.: A Statistical Test Suite for Random and Pseudorandom Number Generators for Cryptographic Applications. NIST Special Publication, 800–22 (2001)
55. Marsaglia, G.: Diehard, A Battery of Tests for Random Number Generators (1997)
56. Norouzi, B., Seyedzadeh, S.M., Mirzakuchaki, S., Mosavi, M.R.: A novel image encryption based on row-column, masking and main diffusion processes with hyper chaos. Multimed. Tools Appl. (2013). doi:10.1007/s11042-013-1699-y
57. Norouzi, B., Mirzakuchaki, S., Seyedzadeh, S.M., Mosavi, M.R.: A simple, sensitive and secure image encryption algorithm based on hyperchaotic system with only one round diffusion process. Multimed. Tools Appl. (2012). doi:10.1007/s11042-012-1292-9